

Lekcja 11 - Spark / PySpark Foundations

2026-09-04

Data Engineering Course

Spark / PySpark Foundations

Kiedy dane są za duże na Polars?
Bronze → Silver transform na PySpark.

1. Dlaczego Spark
2. Architektura i słownik
3. Lazy evaluation
4. Bronze -> Silver na DataFrame API
5. Partitioning, write i jakość
6. Najczęstsze pułapki
7. Homework + interview

■ Learning goals

Po tej lekcji umiesz:

```
wyjaśnić kiedy i po co Spark,  
odróżnić transformation vs action,  
zbudować prosty Bronze -> Silver job,  
zrobić idempotentny zapis partitioned,  
uniknąć 4 typowych błędów początkujących.
```

■ Problem biznesowy

Masz pipeline w Polars. Działa.

Ale skala rośnie:

```
Wczoraj: 2 GB  
Dziś:    500 GB  
Jutro:   5 TB
```

To nie jest problem kodu. To jest problem architektury wykonania.

■ First principles

Jedna maszyna ma limit RAM.
Disk i network są dużo wolniejsze niż RAM.

Gdy dataset > RAM:

- rośnie spill,
- rośnie czas,
- rośnie ryzyko awarii.

Spark rozkłada pracę na wiele executorów.

■ Polars vs PySpark

```
Polars = single-node, bardzo szybki lokalnie  
PySpark = distributed, skala klastra
```

Reguła praktyczna:

```
start: Polars  
scale: PySpark gdy data przekracza możliwości 1  
maszyny
```

■ Co to jest Spark

Spark to engine, który:

```
czyta dane,  
buduje plan,  
optymalizuje plan,  
wykonuje tasks na partycjach,  
zapisuje wynik.
```

Ty opisujesz transformacje. Spark decyduje jak je wykonać.

Driver

-> plan i koordynacja

Cluster manager

-> przydział zasobów

Executors

-> realna praca na danych

Kod PySpark piszesz raz, a Spark rozsyła obliczenia.

■ Słownik 4 pojęć

```
Job      = uruchomienie po action  
Stage    = część joba między shuffle  
Task     = praca na jednej partycji  
Partition = logiczny fragment danych
```

Bez tego słownika trudno czytać Spark UI.

```
from pyspark.sql import SparkSession

spark = (
    SparkSession.builder
    .appName("OrdersPipeline")
    .master("local[*]")
    .getOrCreate()
)
```

Praktyka:

- jedna sesja na job,
- `getOrCreate()` zamiast ręcznego `SparkContext`.

■ Ingest Bronze

```
df_bronze = (  
    spark.read  
    .option("inferSchema", "true")  
    .json("data/bronze/orders/")  
)
```

Bronze to raw layer. Tu nie „ulepszamy” biznesu.

■ Lazy evaluation

```
df1 = spark.read.json("data/orders.json")  
df2 = df1.filter(df1.status == "completed")  
df3 = df2.select("order_id", "total_amount")
```

Na tym etapie Spark jeszcze nie wykonuje obliczeń.

■ Action uruchamia job

```
df3.count()  
df3.show(5)  
df3.write.parquet("output/")
```

Dopiero action triggeruje wykonanie planu.

To ważne dla debugowania i performance.

■ Narrow vs wide

Narrow: filter/select/withColumn

-> zwykle bez shuffle

Wide: groupBy/join/distinct

-> zwykle shuffle

Wide operacje kosztują więcej.

■ DataFrame API mapowanie

```
Polars    -> PySpark
-----
filter    -> filter
with_columns -> withColumn
group_by  -> groupBy
```

Mental model zostaje ten sam. Zmienia się składnia.

■ Bronze -> Silver transform

```
from pyspark.sql import functions as F

df_silver = (
    df_bronze
    .withColumn("status", F.trim(F.lower(F.col("
status"))))
    .withColumn("total_amount", F.col("
total_amount").cast("double"))
    .filter(F.col("order_id").isNotNull())
)
```

Silver = normalization + basic data quality.

■ Silver quality contract

W tej lekcji wymagamy:

```
order_id != null  
status   = lowercase + trimmed  
total_amount = double
```

To minimum, które daje stabilny input do Gold.

■ Write Parquet partitioned

```
(  
    df_silver.write  
        .mode("overwrite")  
        .partitionBy("status")  
        .parquet("output/silver/orders/")  
)
```

`mode("overwrite")` pomaga zrobić rerun bez śmieci.

■ Dlaczego `partitionBy`

```
WHERE status = 'completed'  
-> Spark czyta tylko folder status=completed
```

To nazywa się `partition pruning`.

Mniej skanowanych plików = szybsze query.

■ Idempotencja

Dobrze napisany job:

```
run #1 -> wynik X  
run #2 -> ten sam wynik X
```

W tej lekcji idempotencję osiągasz przez:

- deterministyczne transformacje,
- jawny `mode("overwrite")`.

■ Co sprawdzasz po runie

Checklist operacyjny:

- 1) count rekordów wejścia/wyjścia
- 2) printSchema po Silver
- 3) foldery partycji istnieją
- 4) brak null order_id

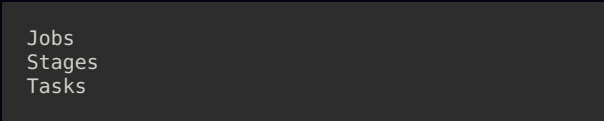
■ 4 pułapki początkujących

- 1) `collect()` na dużym df
- 2) brak `cast` -> typy "string everywhere"
- 3) brak `partitionBy` przy `write`
- 4) brak `overwrite` i śmieci po rerunie

To są realne błędy z projektów juniorskich.

■ Spark UI: pierwszy kontakt

Patrzysz na:



Jobs
Stages
Tasks

Jeśli stage trwa długo, szukasz:

- shuffle,
- skew,
- za dużo danych po drodze.

■ Demo flow na zajęciach

1. Read Bronze
2. show schema
3. Transform Silver
4. Write partitioned
5. Read back i validate count

Każdy krok ma 1 cel i 1 check.

■ Mini quiz

Q1: Czy filter uruchamia job?	(nie)
Q2: Co uruchamia job?	(action)
Q3: Co daje partitionBy(status)?	(pruning)
Q4: Co jest celem Silver?	(clean + contract)

■ Interview answer 1

Pytanie:

Kiedy wybrałbyś PySpark zamiast Polars?

Odpowiedź:

When data exceeds single-node memory limits,
when cluster execution is required,
or when the platform standard is Spark.
Polars is great locally, Spark wins on distributed
scale.

Pytanie:

What is lazy evaluation in Spark?

Odpowiedź:

Transformations build a plan but do not execute immediately.
Execution starts on actions like count, show, write.
This allows Spark to optimize the full DAG before running.

■ Homework

Napisz `spark_bronze_to_silver.py`:

- 1) SparkSession
- 2) read raw JSON
- 3) normalize + cast + filter null
- 4) write parquet partitioned
- 5) minimum 3 testy

■ Definition of done

```
[ ] pipeline działa 2x i wynik jest taki sam  
[ ] testy przechodzą  
[ ] schema Silver jest poprawna  
[ ] partycje status=* są utworzone
```

To jest pełny baseline pod kolejne lekcje.

